DEVSECOPS FINAL CAPSTONE · SESSION 21

ClinicFlow

Clinic appointment platform with a full path from code → CI/CD → secure images → AWS EKS → GitOps → live metrics.

FastAPI · React · Postgres

GitHub Actions · Trivy · GHCR

Terraform · EKS · Helm · Argo CD

Prometheus · Grafana

AGENDA

What we'll cover

From product intent through every DevOps module the rubric grades.

1. **01** Problem & product overview
2. **02** Features & API surface
3. **03** Tech stack
4. **04** End-to-end architecture
5. **05** Repository layout
6. **06** CI/CD pipeline
7. **07** DevSecOps / Trivy
8. **08** Terraform / AWS
9. **09** Kubernetes + Helm
10. **10** Argo CD GitOps
11. **11** Observability
12. **12** Cost control & evidence
13. **13** Key learnings
14. **14** Demo & submission

Why ClinicFlow?

Clinics need a simple way to book visits, track priority/status, and operate the same app from a laptop to Kubernetes — with security and observability baked in.

Product goals

- Book and manage patient appointments (CRUD)
- Track priority (HIGH / MEDIUM / …) and status
- Stats for a quick operational snapshot
- Usable React UI + OpenAPI docs
- Health / ready probes for orchestration

Capstone goals

- Original domain (not a TaskBoard clone)
- Prove M1–M10: app → tests → Git → Docker → CI → Trivy → Terraform → K8s → metrics → presentation
- Immutable SHA-tagged images in GHCR
- Real AWS VPC + EKS, then destroy for cost
- Prometheus UP + Grafana live panels

02 · APPLICATION

Features & API surface

CRUD	4+	Alembic	/metrics
APPOINTMENTS	REST VERBS	MIGRATIONS	PROMETHEUS

METHOD	PATH	PURPOSE
GET / POST	/api/appointments	List & create visits
GET / PUT / DELETE	/api/appointments/{id}	Read, update, cancel
GET	/api/appointments/stats	Counts by status / priority
GET	/health · /ready · /metrics	Liveness, readiness, scrape

Local URLs: UI :3000 · Swagger :8000/docs · Compose one-command bring-up.

03 · TECH STACK

Tools at every layer

App

- React + Vite
- Nginx (non-root)
- Python 3.12 + FastAPI
- SQLAlchemy + Alembic
- PostgreSQL 16

Quality & packages

- Pytest (SQLite test DB)
- Docker multi-stage builds
- Docker Compose
- Non-root containers
- GitHub public repo

Delivery & cloud

- GitHub Actions → GHCR
- Trivy HIGH/CRITICAL gate
- Terraform VPC + EKS
- Helm + Ingress + HPA
- Argo CD · Prom · Grafana

Region for infra demo: **ap-south-1 (Mumbai)** · Images: **ghcr.io/kushals256/clinicflow-***

04 · ARCHITECTURE

End-to-end delivery path

```
Developer → GitHub → Actions (pytest · build · Trivy · push SHA)
        → GHCR images
        → Terraform (VPC + EKS)
        → Argo CD / Helm → Frontend + Backend → PostgreSQL
        → /metrics → Prometheus → Grafana
```

Request path (K8s)

- **/** → Ingress → Frontend Service → UI pods
- **/api** → Ingress → Backend Service → API pods
- Backend → Postgres Service
- ServiceMonitor / scrape job → Prometheus

Local path (Compose)

- Browser :3000 → Nginx → proxies /api
- Backend :8000 → Postgres :5432
- Optional Prom :9091 · Grafana :3300
- Same /metrics contract as cluster

05 · REPOSITORY

How the monorepo is organized

Application

- **frontend/** — React UI + Dockerfile
- **backend/** — FastAPI, Alembic, tests
- **docker-compose.yml** — local stack
- **docker-compose.monitoring.yml** — Prom/Grafana

Platform

- **terraform/** — VPC + EKS modules
- **helm/clinicflow/** — chart + values
- **k8s/** — namespace bootstrap
- **argocd/** — Application CR
- **monitoring/** — values + dashboard
- **.github/workflows/** — CI/CD

> **demo/** holds slides, screenshots (M1–M9 evidence), and the mandatory presentation video.

06 · CI/CD

GitHub Actions → GHCR

Workflow `.github/workflows/ci-cd.yml` · triggers on push/PR to **main**.

JOB 1
Test
Python 3.12 · pytest
Node 22 · npm build

→

JOB 2
Build
docker build backend
+ frontend (tag=SHA)

→

GATE
Trivy
HIGH/CRITICAL
--exit-code 1

→

PUBLISH
GHCR
Push only if scans pass
immutable SHA tags

IMAGE	TAG STRATEGY	WHEN
ghcr.io/kushals256/clinicflow-backend	<git-sha>	push to main after green Trivy
ghcr.io/kushals256/clinicflow-frontend	<git-sha>	push to main after green Trivy

Trivy as a hard quality gate

Policy in CI

- Scan both container images
- `--severity HIGH,CRITICAL`
- `--ignore-unfixed`
- `--exit-code 1` → fail the job
- No push to GHCR on findings

Real remediation example

- First run failed on Starlette 0.41.3
- CVEs: CVE-2025-62727, CVE-2026-48818, CVE-2026-54283
- Fix: upgrade FastAPI → 0.143.2
- Pulls Starlette ≥ 1.3.1
- Pipeline returned to green

> Security is a gate before registry publish — not a slideware checkbox.

08 · INFRASTRUCTURE AS CODE

Terraform → VPC + EKS

What we provision

- VPC **clinicflow-vpc** · 10.20.0.0/16
- 2 public + 2 private subnets (multi-AZ)
- Single NAT gateway
- EKS **clinicflow-eks** · K8s 1.31
- Managed node group: t3.medium · desired 2
- Region: **ap-south-1**

Workflow & evidence

- `terraform init / plan / apply`
- AWS Console: VPC + EKS ACTIVE
- `aws eks update-kubeconfig`
- **terraform destroy** after screenshots
- No AWS keys in Git
- Screenshots in `demo/screenshots/`

Modules: terraform-aws-modules/vpc + eks — reproducible, destroyable infra.

09 · KUBERNETES + HELM

Packaging ClinicFlow for the cluster

Workloads

- Frontend Deployment (≥2)
- Backend Deployment (≥2)
- Demo Postgres
- ClusterIP Services

Edge & scale

- Ingress: / → UI
- Ingress: /api → API
- NLB / LB in front
- HPA on CPU (2→6)

Ops

- Namespace: clinicflow
- values-dev / values-prod
- SHA image tags from CI
- ServiceMonitor enabled

Verify: `kubectl get pods,svc,ingress,hpa -n clinicflow · helm list -n clinicflow`

Argo CD keeps the cluster honest

Application spec

- Source: github.com/kushals256/clinicflow
- Path: `helm/clinicflow`
- Revision: `main`
- Destination: namespace `clinicflow`
- Automated sync + prune + selfHeal

Why it matters

- Git is the source of truth
- Cluster drifts get healed
- Image tags updated from CI SHAs
- UI shows **Synced / Healthy**
- Required clip in the demo video

Manifest: `argocd/application.yaml` · Evidence: `demo/screenshots/25-argocd-synced.png`

Metrics that graders (and ops) can see

EMIT
/metrics
FastAPI counters & latency histograms

→

SCRAPE
Prometheus
ServiceMonitor or static job · target UP

→

VISUALIZE
Grafana
HTTP req/s + p95 latency panels

On EKS

- kube-prometheus-stack Helm chart
- values in `monitoring/`
- Dashboard ConfigMap / JSON
- Port-forward Prom + Grafana

Local (no AWS burn)

- Compose monitoring overlay
- Prometheus :9091/targets
- Grafana :3300 · admin/clinicflow
- Same metric names & dashboard

12 · EVIDENCE & COST CONTROL

Prove it, then tear it down

Screenshot pack (demo/screenshots)

- pytest, Compose, non-root users
- Green Actions + GHCR SHA tags
- Trivy log · terraform plan/apply/destroy
- Console VPC + EKS ACTIVE
- kubectl / helm / ingress / Argo
- Prometheus UP · Grafana panels

Cost discipline

- EKS + NAT cost adds up fast
- Capture Console + kubectl evidence
- Run **terraform destroy** same day
- Document destroy completion
- Keep Prom/Grafana demo local afterward
- Rotate any exposed IAM keys

> Rubric: runaway AWS bills are the student's responsibility — destroy is part of the grade story.

13 · KEY LEARNINGS

What this project taught

Engineering

- Fail closed on tests before images exist
- Trivy turns CVEs into concrete upgrades
- SHA tags beat floating `latest`
- Helm + values keep env differences explicit
- ServiceMonitor ties apps to Prometheus cleanly

Operations & craft

- IaC must be destroyable, not just creatable
- GitOps reduces “snowflake cluster” drift
- Metrics need traffic — generate load to demo
- Original domain + shared DevOps patterns
- Evidence screenshots matter as much as code

Demo walkthrough & submission

Live next: codebase → CI push → Terraform/AWS evidence → Argo Synced → kubectl/Helm → Prometheus UP → Grafana panels.

Repo & docs

- github.com/kushals256/clinicflow
- README — full technical docs
- GRADING.md · SUBMISSION.md
- demo/slides.pdf · demo/screenshots/

Submit

- Form: forms.gle/XWAP1vAumDJgAPM1A
- Deadline: Sun 19 Oct 2026, 11:59 PM IST
- Video MP4 in `demo/` (mandatory)
- Public repo link in the form